

实验六 数据库查询性能调优实验报告

- 题目：数据库查询性能调优实验
- 姓名：张昕跃
- 学号：2023202300
- 实验环境：Ubuntu 24.04.3 LTS, PostgreSQL 16.11 (Ubuntu 16.11-0ubuntu0.24.04.1)
- 数据集：TPC-H (10GB)

说明：所有用到的代码为当前压缩包中sql_scripts文件夹中的*.sql文件，对于脚本输出，为results文件夹中对应文件名+output.txt文件。
cleanup.sql代码为清除任务2中建立的索引等，恢复到数据库初始状态。
为方便阅读，我把实验总结和问题建议部分提前了，并把结果分析合并在了步骤中。

一、实验总结和问题建议

遇到哪些问题，如何解决。对本实验的改进建议（必答，算分！）

问题上主要有如下几个：

1. 原来的参数会导致用脚本的话，跑q17耗时数小时都没有结果，一度以为是我自己机器的问题，后来做了任务3调了系统参数就成功了，大大提高了效率。
2. 使用AI帮忙做查询优化的时候，一开始效果很差，后来才给出一些性能较大提升的方法，因此**不能轻信大模型**
3. 对于任务三调试参数，不太建议一次性调整很多，不一定是正面提升，反而会降低性能，可以考虑一个参数一个参数的试，控制变量观察结果，但是耗时比较长
4. 因此建议每个任务都写一个脚本来做，用.sh+.sql，我一开始懒得写，因此浪费了很多时间，后来用shell脚本跑，跑的时候可以去做别的事情节省时间。
5. 在优化了查询之后，已经有大幅提升了，其实再改参数效果就不是特别明显。
6. 对于postgresql的一些步骤和mysql不太一样，不过文档中附的另外一个链接讲解很详细。
7. 假如加了聚簇的话，其实不重新导入数据就没法恢复了，最好还是先不要考虑这个优化，或者在优化之前一定先跑三次留下初始结果！

改进建议：

1. 一定要说明q17不改系统参数查不出来的问题，在这个上面浪费了很多时间！！
2. 说明一下有些更改是不可逆的，除非重新创建数据，不然很有可能出现解释不了的实验结果，或者无法复现初始结果了。
3. 让大家自己写一些脚本去测试，而不是每一次都在命令行去跑sql然后手动记录终端的结果。

二、实验步骤与结果分析

本次实验主要针对 TPC-H 的 **Q5 (Local Supplier Volume Query)** 和 **Q9 (Product Type Profit Measure Query)** 进行性能分析与优化。

1. 任务1：使用DBMS工具分析查询性能瓶颈

1.1 实验操作

使用 `EXPLAIN (ANALYZE, BUFFERS)` 命令对原始查询进行分析。

脚本： `analysis_q5.sql` 和 `analysis_q9.sql`

核心命令示例 (Q5)：

```
EXPLAIN (ANALYZE, BUFFERS)
select
  n_name,
  sum(l_extendedprice * (1 - l_discount)) as revenue
from
  customer, orders, lineitem, supplier, nation, region
where
  c_custkey = o_custkey
  and l_orderkey = o_orderkey
  and l_suppkey = s_suppkey
  and c_nationkey = s_nationkey
  and s_nationkey = n_nationkey
  and n_regionkey = r_regionkey
  and r_name = 'ASIA'
  and o_orderdate >= date '1994-01-01'
  and o_orderdate < date '1994-01-01' + interval '1 year'
group by n_name
order by revenue desc;
```

1.2 Q5 性能分析

通过执行 `analysis_q5.sql`，我们观察到原始执行计划如下：

- **总执行时间:** 三次基准测试平均约 **59,021 ms (59s)**。
- **全表扫描:** `Parallel Seq Scan on lineitem` 扫描了约 2000 万行数据 (`actual rows=19,995,351`)。
- **Join 策略:** 使用了大量的 `Parallel Hash Join`。
- **瓶颈:**
 - **I/O 开销:** `lineitem` 表的全表扫描是主要瓶颈。
 - **Hash Join 代价:** `lineitem` 与 `supplier`, `orders` 的 Hash Join 消耗了大量计算资源 (`actual time` 占比很大)。
 - **Buffer 命中:** 虽然 Shared Hit 较高，但在没有索引的情况下，必须读取大量页面 (`read=1124983`)。

1.3 Q9 性能分析

通过执行 `analysis_q9.sql`，原始执行计划显示：

- 总执行时间: 约 101,930 ms (102s)。
- 执行与扫描:
 - Parallel Seq Scan on lineitem: 再次扫描了 2000 万行数据。
 - Parallel Seq Scan on part: 扫描了 7.5 万行, 过滤条件 `p_name like '%green%'` 过滤掉了绝大部分数据, 但仍需全表扫描。
- 复杂连接: lineitem 与 partsupp 的连接以及后续与 orders 的连接是性能热点。
- 瓶颈:
 - Hash Aggregate: 分组聚合操作 (Partial HashAggregate) 耗时较长。
 - 大表 Join: 多层 Hash Join 导致内存和 CPU 压力巨大。

2. 任务2: 使用查询优化技术提升查询性能

2.1 优化策略

我采用 建立 B-Tree 索引 的方法来优化 Scan 和 Join 路径。

脚本: `optimize_q5.sql` 和 `optimize_q9.sql`

Q5 索引优化核心代码:

```
-- Covering Index for Orders: 支持 Join + 时间过滤
CREATE INDEX IF NOT EXISTS idx_orders_q5_cover
  ON orders(o_custkey, o_orderdate)
  INCLUDE (o_orderkey);

-- Covering Index for Lineitem: 完全覆盖所有需要字段
CREATE INDEX IF NOT EXISTS idx_lineitem_q5_cover
  ON lineitem(l_orderkey, l_suppkey)
  INCLUDE (l_extendedprice, l_discount);

-- Customer Nation 外键索引
CREATE INDEX IF NOT EXISTS idx_cust_nation ON customer(c_nationkey);
```

Q9 索引优化核心代码:

```
-- PartSupp Covering Index
CREATE INDEX IF NOT EXISTS idx_partsupp_q9_cover
  ON partsupp(ps_partkey, ps_suppkey)
  INCLUDE (ps_supplycost);

-- Lineitem Covering Index
CREATE INDEX IF NOT EXISTS idx_lineitem_q9_cover
  ON lineitem(l_partkey, l_suppkey, l_orderkey)
  INCLUDE (l_extendedprice, l_discount, l_quantity);

-- Orders Year Index
CREATE INDEX IF NOT EXISTS idx_orders_year ON orders(o_orderdate);
```

```
-- Supplier Nation Index
CREATE INDEX IF NOT EXISTS idx_supp_nation ON supplier(s_nationkey);
```

优化要点：

- 使用 `INCLUDE` 子句创建覆盖索引，实现 Index Only Scan。
- 针对 Q9 的 `p_name LIKE '%green%'` 过滤，由于通配符前缀无法使用 B-Tree 索引，故不为 `part(p_name)` 建索引。

2.2 优化结果对比 (单位: ms)

查询	原始延迟 (Baseline)	优化后延迟 (Optimized)	降低百分比	备注
Q5	59021 ms (60476, 55936, 60650)	2591 ms (3852, 1861, 2060)	95.6%	Covering Index 实现 Index Only Scan
Q9	87298 ms (79086, 94313, 88495)	9833 ms (8828, 11643, 9028)	88.7%	Covering Index (partsupp, lineitem) 加速 Join

3. 任务3：调整数据库参数配置提升查询性能

3.1 参数调整

为适应 OLAP 复杂查询，我们调整了 PostgreSQL 配置，完整参数可见 `apply_parameters.sql`

脚本： `modifyparas_q5.sql` 和 `modifyparas_q9.sql`

参数配置代码：

```
-- 1. 增加每个操作的可用内存
SET work_mem = '256MB';
-- 2. 针对 SSD 调整随机 I/O 成本估算
SET random_page_cost = 1.1;
-- 3. 告知优化器系统缓存大小
SET effective_cache_size = '12GB';
-- 4. 设置并发 I/O 能力 (SSD)
SET effective_io_concurrency = 200;
-- 5. 增加并行 Worker 数量
SET max_parallel_workers_per_gather = 16;
-- 6. 关闭 JIT 编译
SET jit = off;
```

参数说明：

1. `work_mem = 256MB`: 保证 Hash Join 和 Sort 操作在内存中完成，避免磁盘交换。
2. `random_page_cost = 1.1`: 降低随机读取成本估值，鼓励优化器使用索引。
3. `effective_cache_size = 12GB`: 告知优化器可用文件缓存大小，辅助决策。

4. `effective_io_concurrency = 200`: 提高 SSD 的并发 I/O 能力。
5. `max_parallel_workers_per_gather = 16`: 增加并行 Worker 数量, 极大加速全表扫描 (Seq Scan)。
6. `jit = off`: 关闭即时编译, 节省短查询的编译开销 (约 5s)。

3.2 优化结果对比 (仅参数调整 vs 原始)

查询	原始配置延迟	参数调优后延迟	降低百分比	原因分析
Q5	59021 ms	19779 ms (14275, 24100, 20961)	66.5%	并行度提高(16 workers)加速了 Hash Join
Q9	87298 ms	54664 ms (54211, 52718, 57063)	37.4%	并行扫描 I/O 吞吐提升, 但仍受限于全表扫描

3.3 [选做] 更改参数配置后, 利用提供的脚本tpch-benchmark-olap.py测试更改前后整体22条查询的总延迟, 并计算相关指标。

优化之间, 跑到查询q17就失败了, 卡住跑了两个小时都没有跑完这个查询, 但优化完参数之后, 仅仅只用了两秒。所有优化完以后的查询时间如下, 文件是 `all_optimized.txt`, 具体log可见 `log_all_optimized.txt`, 剩下两个 `all_origin` 是原先的运行日志, 但是q17卡住了。

```
=== SQL Execution Summary ===
Query d1.sql      : 14.234136 seconds
Query d2.sql      : 5.158898 seconds
Query d3.sql      : 39.672429 seconds
Query d4.sql      : 38.059308 seconds
Query d5.sql      : 16.578713 seconds
Query d6.sql      : 36.655463 seconds
Query d7.sql      : 62.989494 seconds
Query d8.sql      : 11.136590 seconds
Query d9.sql      : 11.910765 seconds
Query d10.sql     : 38.282987 seconds
Query d11.sql     : 6.160415 seconds
Query d12.sql     : 98.091373 seconds
Query d13.sql     : 31.588269 seconds
Query d14.sql     : 241.944314 seconds
Query d15.sql     : 16.066582 seconds
Query d16.sql     : 8.531771 seconds
Query d17.sql     : 2.824582 seconds
Query d18.sql     : 284.231160 seconds
Query d19.sql     : 10.538403 seconds
Query d20.sql     : 99.933890 seconds
Query d21.sql     : 87.697110 seconds
Query d22.sql     : 17.887374 seconds

Total Execution Time for Queries: 1180.174025 seconds
Script Total Time (including overhead): 1180.236700 seconds
=== Benchmark Completed ===
```

4. 任务4：多种方法结合提升查询性能

4.1 综合测试 (针对 Q9)

我们通过脚本 `combined_q9.sql` 同时应用了索引优化和参数调优。该脚本包含：

优化策略：

1. 设置会话参数 (与任务3一致):
 - `work_mem = 256MB`
 - `random_page_cost = 1.1`
 - `effective_cache_size = 12GB`
 - `effective_io_concurrency = 200`
 - `max_parallel_workers_per_gather = 16`
 - `jit = off`
2. 创建 **Covering Indexes** (与任务2一致):
 - `idx_partsupp_q9_cover` (ps_partkey, ps_suppkey) INCLUDE (ps_supplycost)
 - `idx_lineitem_q9_cover` (l_partkey, l_suppkey, l_orderkey) INCLUDE (l_extendedprice, l_discount, l_quantity)
 - `idx_orders_year` (o_orderdate)
 - `idx_supp_nation` (s_nationkey)

协同效应分析：

- 索引 将 Join 算法从 Hash Join 转化为高效的 Nested Loop + Index Only Scan，消除了回表开销。
- 并行参数 进一步加速了仍需全表扫描的部分（如 `Parallel Seq Scan on part` 用于 `p_name LIKE '%green%'` 过滤）以及 Hash Aggregate 聚合操作。
- 两者结合使得 Q9 从原始的 **87.3秒** 降至 **8.8秒**，实现了 **接近 90%** 的性能提升。

4.2 最终结果

优化阶段	延迟 (Latency)	相比原始提升
1. 原始状态 (Baseline)	87298 ms	-
2. 仅查询优化 (SQL Only)	9833 ms (8828, 11643, 9028)	88.7%
3. 仅参数调优 (Params Only)	54664 ms (54211, 52718, 57063)	37.4%
4. 综合优化 (Combined)	8772 ms (9114, 8423, 8778)	89.9%

4.3 结论

查询优化 (Indexing) 提供了数量级的性能提升，是优化的基础。参数调优 则锦上添花，确保了硬件资源的充分利用。两者结合能达到最佳性能。